Data encryption

Encryption protects data in transit (moving across a network) and at rest (stored on a device or server). For data in transit, TLS 1.3 is the current standard for TCP connections, and it removed many older, weaker options. For devices that use UDP, such as those running CoAP, DTLS 1.3 offers the same protection and was designed with constrained devices in mind. Where messages pass through several hops, object-level protection such as OSCORE can keep them encrypted from device to final receiver.

Most modern protocols rely on authenticated encryption, which hides the data and detects tampering in a single step. Table 3 lists the main building blocks and where they fit.

For data at rest, the main rule is simple: never leave secrets in plain text. Keys and credentials belong in a protected store, such as the secret store included with EdgeX Foundry or the secure storage offered by the chip itself. Encrypting sensitive files on flash adds another layer if a device is stolen or its memory is read out.

Open source frameworks compared

The frameworks fall into two groups. Cryptographic libraries provide the building blocks, such as TLS, ciphers and certificates. Platforms and tools use those libraries to deliver complete features such as secure messaging, device management and secure updates. Table 4 compares four widely used libraries.

The licence matters more than it might seem. A permissive licence such as Apache 2.0 lets a company ship a product without opening its own code, while the GPL does not. Teams should check this early, before they build around a library.

Beyond libraries, several platforms bring everything together. Table 5 summarises five of them, and Figure 3 shows one of the most important protections, Secure Boot, which stops a device from running firmware that has not been properly signed.

Practical guidance

Choosing a framework is only half the job. Most real-world breaches come from avoidable mistakes, not broken cryptography. A few habits help a great deal. Give every device its own unique credentials instead of one shared key baked into the firmware. Never switch off certificate checking just to make a test work and then forget to switch it back on. Keep the libraries updated, because TLS code is a frequent target for vulnerabilities. Finally, plan for firmware updates

Table 3: Encryption building blocks used in open source IoT stacks

Primitive	Role	Best suited for
AES-GCM	Authenticated encryption	Chips with AES hardware acceleration
ChaCha20-Poly1305	Authenticated encryption	Small CPUs without AES hardware
ECDHE (P-256, X25519)	Key agreement per session	Forward secrecy on constrained devices
ECDSA, Ed25519	Digital signatures	Certificates and firmware signing
Ascon	Lightweight authenticated encryption	Very small devices (selected by NIST in 2023)

Table 4: Open source cryptographic and TLS libraries

Library	Licence	Strength	Typical use
Mbed TLS	Apache-2.0 or GPL-2.0-or-later [4]	Small, readable code	Microcontrollers, Zephyr
wolfSSL	GPLv3 or commercial	Compact, many ports	Embedded and RTOS devices
OpenSSL	Apache-2.0 (version 3 onwards)	Feature-rich, widely tested	Gateways, Linux devices
libsodium	ISC	Simple, hard-to-misuse API	Custom protocols, apps

Table 5: Open source platforms and tools for IoT security

Framework	Role	Security features
Eclipse Mosquitto	MQTT message broker	TLS, client certificates, access control lists
EdgeX Foundry	Edge computing platform	Secret store and API gateway for services
MCUboot	Secure bootloader	Signed images, rollback protection, safe updates
Zephyr RTOS	Real-time operating system	Integrates TLS libraries and MCUboot
Eclipse Leshan	LwM2M device management	DTLS with PSK, raw public key or certificates